STUDY.md — 학습 정본 (이 문서 하나로 전 분야·순서·완주)

> **한 줄**: `~/Desktop/seungwoo` 코퍼스(~40 레포)를 *직접 재구성하며* 익히는 **단일 완본**이다 — 커밋을 **L3/L2/L1**로 나눠 **L3만 손으로 다시 만들고**, AI에게 시켜 **틀린 곳을 잡는다**(리뷰). 코드 *재현*이 아니라 *판단력*. **무엇을·어떤 순서로·어떻게 도는지 전부 여기 있다.**
>
> **읽는 법**: 처음이면 Part 1→2를 읽고(왜·한 레포 도는 법), 네가 들어갈 분야를 Part 4에서 펴고, 순서는 Part 5(두 순서의 빠른 대조는 **Part 0**), 끝은 Part 6~7. 시간이 없으면 **Part 5의 경로 A 척추**만 따라가도 완주된다.
>
> ℹ️ 이 문서는 구 `SEQUENCE.md` · `LEARNING.md` · `mobile-track.md` 를 **흡수한 정본**이다. 그 셋은 외부 §참조 호환을 위해 동결 보존된 레거시다 — **여기만 읽으면 된다.** 전 레포 L3 색인(면접용)만 INTERVIEW.md를 부록으로 둔다.

목차

- **Part 0 · 노트-의존 vs 학습곡선 순서** — 두 순서의 정의 · 트랙별 대조표 · 갈리는 지점
- **Part 1 · 왜 이렇게 배우나** — AI 시대의 전제, 두 근육, 3원칙
- **Part 2 · 한 레포 도는 법** — 아티팩트 지도 · 커밋 3분류 · 보편 루프 Step 0~5 · dual-form
- **Part 3 · 레포 장르 6종** — "재구성"의 대상이 다르다 + 6개 워크드 예시
- **Part 4 · 분야별 학습 가이드** — 11개 분야 카드(대장·스택·notes 순서·리듬·L3·검증·함정)
- **Part 5 · 전체 순서** — 의존 사슬 3 · 경로 프리셋 A~F · 42 병렬(F)
- **Part 6 · 캡스톤** — sportsbook 9레포 · Tier-1 8노트 · 모바일 클라이언트 · API 계약
- **Part 7 · 완주 & 면접** — 완료 기준 · 경로 A 체크리스트 · 12 주제축 · 첫 걸음
- **부록 · 스택 → 대장 참조표**

Part 0 · 노트-의존 순서 vs 학습곡선 순서 (빠른 참조)

이 코퍼스에는 **두 순서**가 있다 — 보통 겹치지만 갈리는 지점이 있으니, 트랙에 들어가기 전 어느 쪽을 따를지 정하라.

- ① **노트-의존 = "대장-first"** — 스택을 처음 보면 그 스택의 **대장(가장 깊은 선행 노트)**부터 깊게 정독(부록 참조표·Part 4). *노트가 서로를 전제하는 깊이* 순서.
- ② **학습곡선 = "경로/경사 동선"** — 목표·난이도에 맞춘 **프로젝트 순회**(Part 5 경로 A~F). *난이도가 부드럽게 오르도록* 배치.

강한 의존 사슬 3개(C→C++→시스템 · TS→프론트 · 모바일 foundations→reliability)**에서는 둘이 일치**한다. 갈리는 건 두 경우뿐: **(가)** 대장이 어려운 `*-reliability` 라서 곡선이 그 앞에 부드러운 `*-foundations` 를 **워밍업으로 끼울 때, (나) 자족 입문 노트**(`portfolio`)·**독립 섬**(`stack-sort` =Python)을 의존이 아니라 *교육 논리로* 배치할 때.

트랙	① 노트-의존 (대장-first)	② 학습곡선 (권장 순회)	갈림?
C	`small-shell` (c-language) → 곁가지 노트(서로 독립)	`small-shell` → `format-printer` → `signal-message-bus` → `thread-dining` → `stack-sort`	거의 일치 (곁가지 상대순서·Python은 ②가 배치)
C++ (트랙 =C++98)	`stl-container` (cpp98) → ray(노트 없음)	`stl-container` → `ray-scene-tracer`	일치
C++ (방언 전부)	대장-first: `game-server-reliability` · `irc` 먼저	경사: stl(98)→ray→ `irc` (17)→ `gsf` → `gsr` (20)	**갈림** (Part 4 시스템서버 카드 명시)
백엔드 (JVM)	`backend-foundations` (대장) → `backend-reliability`	`linux-admin` → `backend-foundations` → `backend-reliability` (→ `container-stack` → `sportsbook`)	일치 (단 *Go만*은 대장이 reliability)
백엔드 (Node/TS)	`grounded-travel` (TS 대장) → `chatbot` · `pong`	`grounded-travel` → `chatbot-evaluation` → `pong-pong`	일치

프론트엔드 ★	`grounded` (TS)→ `frontend-reliability` (React 대장)→ `portfolio` (Next 대장)	`grounded` → `frontend-foundations` (zustand)→ `frontend-reliability` → `portfolio-site`	**갈림** — ②가 `foundations` 를 워밍업으로 먼저, `portfolio` 는 자족 입문
모바일	`mobile-reliability` (Kotlin/Flow 대장)	`mobile-foundations` → `mobile-reliability` (사슬 3)	**갈림** — 프론트와 같은 패턴
시스템 서버	`game-server-reliability` (C++20 대장)· `irc` (17)	`gsf` → `gsr` (또는 C++98→17→20 경사)	**갈림** — C++ 방언 행과 동일
인프라 · AI	container-stack · af→ar→capstone	동일	일치 (선형)

> **한 패턴으로 외워라:** `*-foundations` / `*-reliability` 쌍에서 **reliability가 그 스택의 대장**이면 (React·모바일·Go·시스템서버) 노트-의존은 **reliability로 직행**, 곡선은 **foundations를 워밍업으로 먼저** 넣는다. **JVM만 예외**(`backend-foundations` 가 곧 대장이라 일치). 트랙별 상세는 Part 4, 전체 경로는 Part 5.

Part 1 · 왜 이렇게 배우나 (AI 시대의 전제)

AI가 코드를 싸게 찍을수록 값이 오르는 건 **판단력**이다 — 무엇이 옳은지 알고, 틀린 걸 잡고, 결과를 책임지는 것. 그래서 목표는 "코드 재현"이 아니라 두 근육이다:

- **재구성** — 판단이 실린 부분을 직접 만들며 *"내가 진짜 아는가"* 를 시험한다. 벽에 부딪히는 순간이 학습 신호다.
- **리뷰** — AI가 틀린 곳을 잡는다. 답지가 **정답 오라클**이라 이 코퍼스로 바로 훈련된다.

함정: **산문 노트만 다시 쓰는 것.** 유창한 글은 가짜 이해가 숨는 곳이다(4차 품질 런이 사냥한 바로 그 결함). 노트는 재구성의 **산출물**이지 대체물이 아니다 — 벽에 부딪힌 *뒤에* 당신 언어로 고쳐 써야 빌려온 산문이 아니라 얻은 판단이 남는다.

3원칙 (학습 강도를 정하는 잣대):

1. **재유도 > 암송** — "코드를 떠올린다"가 아니라 "빈 파일에서 같은 결정·이유로 재구성한다"로 시험.
2. **빈도가 자동화를 깎는다** — 외우려 들지 말고 반복 인출 상황(직접 만들기)을 세팅한다.
3. **폭발 반경 큰 것만 과투자** — 동시성·트랜잭션·보안·돈 계산은 의도적으로 판다. 나머지는 빈도에 맡긴다.

이 3원칙이 그대로 다음의 **L3 / L2 / L1**(반드시 내재화 / 펼쳐서 / 찾아봐도 됨)로 이어진다.

Part 2 · 한 레포 도는 법

2.1 아티팩트 지도 — 어느 레포를 열든 같다

```
<repo>/
├─ docs/DESIGN.md         ⓪ 설계 의도 — 왜 이 레포·무엇을 만들기로 (들어갈 때 먼저 1회)
├─ notes/                 ① 스택 교과서(깊은 선행 정독, 4부 포맷)
│  └─ reference-impl/     ② 짝 실행체(빌드·실행하며 손에 익힘)
├─ src · *-main …         본 프로젝트 코드 (정답 산출물)
└─ docs/
   ├─ commits/ NNN.md     ③ 답지: 커밋별 완성 코드 + 왜 (끝에 L3/L2/L1 색인)
   ├─ practice/ NNN.md    ④ 문제지: 답지에서 코드 본문을 비운 연습용
   ├─ notes/              ⑤ 온디맨드 어휘(얕은 곁노트 — 막힐 때만)
   └─ reflection/         회고 · 실패 모드 · 변경 비용
```

#	아티팩트	무엇	읽는 법
⓪	`docs/DESIGN.md`	왜 이 레포·무엇·의도적 비범위·구현 경로 개관	**진입 시 먼저 1회.** L3가 어디인지 잡고 들어간다
①	루트 `notes/<stack>.md`	스택 1개당 1노트, **4부 포맷**(개념→본 코드 분석→미니프로젝트→심화)	처음 보는 스택만 **깊게 선행 정독**(대장-first)
②	`notes/reference-impl/`	노트의 짝 실행체(빌드 가능)	한 번 빌드·실행해 손에 익힘
③	`docs/commits/NNN.md` (답지)	비-merge 커밋 1개당 1문서, 완성 코드 + 왜	직접 푼 뒤 **마지막에** 펼쳐 채점
④	`docs/practice/NNN.md` (문제지)	답지에서 본문을 비운 것(계약 노출, 본문 `// TODO`)	**답지 덮고** 직접 채운 뒤 대조(재유도)
⑤	`docs/notes/<cat>/`	얇은 개념 어휘(온디맨드, 일부 레포만)	커밋 따라가다 **막힐 때만**

`notes/` 와 `docs/notes/` 는 정반대 깊이다 (가장 헷갈리는 지점). 루트 `notes/` = 깊게 선행하는 **교과서**(4부), `docs/notes/` = 막힐 때만 꺼내는 **곁노트(용어 사전)**. "더 최근 = 진짜"가 아니라 *역할이 다른 두 계층*이다.

2.2 커밋을 셋으로 나눠라 — 전부 같은 깊이로 돌지 마라

어느 급인지는 답지 끝 **L3/L2/L1**에 다 적혀 있다. 시작 전에 그것부터 보고 분류한다.

급	무엇	어떻게
L3	동시성·트랜잭션·멱등·돈 계산·실패 모드 (코퍼스의 심장)	**손으로 재구성.** 목표 = 코드 재현이 아니라 *백지에서 모든 결정 방어 + 실패 모드 설명*
L2	구조·흐름·책임 분리	답지 펼쳐 설명되면 충분. 재구성은 선택(리뷰 드릴로 변주하기 좋음)
L1	보일러플레이트·CRUD·문법·설정	**읽고 통과.** 다시 치지 마라 — AI가 메우는 영역, 손 연습 가치 낮음

2.3 보편 루프 (The Loop) — 프로젝트 1개 도는 법

Step 0 — 진입 판별. 레포 `docs/DESIGN.md` (⓪) 읽기 → 왜/무엇/설계 의도 파악(L3 위치가 보인다). Part 4의 해당 분야 카드로 스택·대장 확인.

Step 1 — 스택 선행(notes 정독). 루트 `notes/` 를 **분야 카드의 읽기 순서대로**, 처음 보는 스택만 1부(개념)부터 깊게, 아는 스택이면 2부(본 코드 분석)만. 짝 `reference-impl/` 있으면 빌드해 한 번 만진다.

Step 2 — 재구성 ★핵심. `docs/commits/` 를 번호순으로 따라가되 **덮고 직접 만든다.** 커밋 리듬 두 가지:

- **(2A) exercise형 — "문제 계약 / 구현 / 테스트" triplet**: ① 계약 문서 읽기 → ② **여기서 멈추고 덮어, 계약만 보고 직접 구현**(재유도 엔진) → ③ 구현 문서 펼쳐 diff → ④ 테스트 문서로 검증 모델 익히기.
- **(2B) build형 — 순차 기능 커밋**: ① 제목·본문 첫머리만 읽어 이번 증분 목표 파악 → ② **덮고** 직전 상태에서 목표까지 직접 구현 → ③ 커밋 문서·원본 대조.

→ `git show <커밋>` (또는 답지)와 **diff** → 내 결정이 다르면 *왜* 다른지 따진다. **여기가 학습의 본체다.**

Step 3 — 온디맨드 어휘. Step 2에서 어휘가 막히면 그때만 `docs/notes/` (⑤)를 꺼내 읽고 복귀. 처음부터 통독 금지.

Step 4 — 복기(L3/L2/L1 필터). 답지 끝 `## 기억/설명 Level` : **L3** → 문서 덮고 백지 설명/작도, **L2** → 소스·문서 펼쳐 설명, **L1** → 스킵. L3가 백지에서 안 나오면 그 커밋은 **안 끝난 것.**

Step 5 — 회고·검증으로 닫기. `docs/reflection/` 읽거나 채우고, 완료 기준(Part 7)으로 단위를 닫는다.

> **모드 B · 리뷰 드릴**(변주 — 살아남는 직무를 직접 훈련): AI에게 그 커밋을 구현시킨다(문제지의 계약·책임만 주고 답지는 안 줌) → 답지와 **diff** → *AI가 어디서 더 나쁜 결정을 했나 / 실패 모드를 놓쳤나 / API를 환각했나* 를 찾는다. 답지가 채점 오라클이라 정답 보장된 리뷰가 된다.

dual-form 한눈에. 답지(`docs/commits/`)가 정본, 문제지(`docs/practice/`)는 파생이다. 실체 있는 코드 커밋만 문제지를 가진다(마커·순수 문서·순수 타입 정의 커밋은 답지만). 일부 답지의 `## 알려진 소스 결함` 절은 원본의 미수정 결함·수정 방향이니 재구현 때 따른다.

막히는 순서(두 모드 공통): `docs/notes/` 용어집 → 답지의 **해당 `###` 한 절만** → 그래도면 답지 전체.

안티패턴: `000→N` 을 소설처럼 읽고 베껴 쓰기(가장 약한 학습, 재유도 원칙 위배). 반드시 덮고 먼저 시도.

Part 3 · 레포 장르 6종 — 루프는 하나, '재구성'의 대상이 다르다

Part 2의 루프는 모든 레포에 같다. 하지만 **"무엇을 재구성하느냐"는 장르마다 다르다** — 그래서 어떤 레포는 "답지 덮고 코드 채우기"가 헛돈다. 들어가면서 `docs/DESIGN.md`(⓪)가 장르와 L3 위치를 알려준다. 빠른 신호: **답지 수 ≈ 문제지 수면 코드 재구성형, 문제지가 거의 없으면 절차·손계산·규율형이다.**

장르	대표 레포	재구성 대상	"다음으로" 완료 바
① **코드 재구성형** (기본·대다수)	bf·br·sportsbook·웹/TS/모바일/AI	문제지 덮고 **코드 본문**	L3 백지 방어 + 실패 모드
② **절차·설정형**	container-stack	**설정 파일·절차** (문제지 풍부)	백지에서 구조도 + **왜 이 정책**
③ **손계산·분석형**	network-routing-notes	**손으로 재계산**(서브넷/라우팅)	손계산이 검증 스크립트와 일치
④ **따라만들기형**	ray-scene-tracer	빌드를 **따라 만들며**(노트 없음)	빌드 그린 + 체크섬 결정성
⑤ **작업규율 체화형**	linux-admin (M0·첫 레포)	재구성 아님 — **규율을 실제로 지키며 한 바퀴**	규율이 다음 레포에서 그대로 나옴
⑥ **test-first형**	gsf·gsr (시스템서버)	**테스트 먼저 읽고** 구현	테스트가 green

각 장르를 대표 레포로 한 바퀴 돈 예시 — DESIGN ⓪에서 장르를 식별하고 루프를 그 장르에 맞춰 구부린다.

① 코드 재구성형 · `backend-foundations` 1. **DESIGN ⓪ 먼저(1회)** — JVM 대장인 이유·만들기로 한 것(spine/branches/bridge)·구현 경로. "L3가 어디냐"가 보인다. 2. **`notes/` 정독**(Part 4 JVM 카드 순서): 언어 `java-21` → 빌드 `gradle` → 코어 `spring-boot`·`spring-web-mvc` → 기능 `validation`·`security`·`pagination` → 테스트 `junit5`·`testcontainers`. `reference-impl/` 한 번 빌드. 3. **`docs/commits/README` 읽는 순서로 번호 따라간다.** merge·메타는 파일 없이 건너뛰고, 문제지는 실체 있는 커밋에만(예: 005·006·009…). 각 번호: 분류 → 덮고 구현 → 문제지 `## 검증` (`make test-spring-exercise EX=<과제>`) green → 답지 diff. 4. **다음 번호 = 그 급의 완료 바:** L3(bridge `idempotent-create-lite`·`transaction-boundary-lite`) 백지 방어; L2(spine CRUD·`06-testing-slices`) 펼쳐 설명; L1(`01-http-json-basics` 보일러플레이트) 읽고 즉시 다음. 5. 닫으면 INTERVIEW.md의 bf L3 줄 복기 → 다음 레포.

② 절차·설정형 · `container-stack` (42 Inception — WordPress 스택) 1. DESIGN ⓪ → 한 Compose에 nginx(TLS/FastCGI)·WordPress(php-fpm)·MariaDB를 조립하는 게 목표, L3는 *왜 이 구성*. 2. `notes/` 17편을 분야 카드 순서로(셸 → 이미지 → 조립 → 웹 → DB). 3. 문제지 18편(000-017)이 답지와 거의 1:1 — **코드형처럼 덮고 설정을 재구성**하되 채점은 *정적 검증기*(+실제 기동). 4. 완료 바: 설정 외우기가 아니라 **백지에서 헬스체크 체인·secret 분리·진입 경계를 *왜 이렇게*인지 방어**되면 다음.

③ 손계산·분석형 · `network-routing-notes` (노트 없음 — 설계) - 산출물이 바이너리가 아니라 **문서 + 검증 스크립트**(subnet 계산 체크리스트 + Python/shell). 재구성 = 서브넷/라우팅을 **손으로 계산**해 스크립트와 대조. - 문제지 2편(002·003)뿐 — 답지 보기 전 *직접 계산*하고 스크립트로 채점. - 완료 바: 손계산이 스크립트 결과와 일치하고 *왜 이 서브넷/경로*인지 설명되면 다음. (경로 A에선 선택.)

④ 따라만들기형 · `ray-scene-tracer` (노트 없음, 선행 = stl-container) - 새 스택을 익히는 레포가 아니라 **C++ 대장(stl-container)에서 익힌 걸 적용**하는 그래픽스 연습 — 그래서 교과서 노트가 없다. - 문제지 8편을 번호順으로 **따라 만들며** 쌓는다(L3는 적다). 검증 = 출력 **체크섬 결정성**(같은 입력 → 같은 해시). - 완료 바: 빌드 그린 + 체크섬이 답지와 일치하면 다음.

⑤ 작업규율 체화형 · `linux-admin` (M0 — 모든 경로의 첫 레포) 1. DESIGN ⓪ → L3가 코드가 아니라 *규율·정책*(Git·Conventional Commits·Markdown + Born2beroot Linux 운영: 사용자·SSH·sudo). 2. `notes/` 4편(`git-cli` → `conventional-commits` → `github-workflow` → `markdown`)을 읽고 **바로 적용** — 이 레포를 도는 동안 *네 커밋 자체가* 규율 연습이다. 3. 답지 12편 중 문제지는 2편(003·004)뿐 — 나머지는 순수 문서·설정이라 비울 코드가 없다. **덮어 채우는 대신, 절차를 읽고 네 환경에서 실제로 한 번 한다.** 4. 완료 바: 규율이 *다음 커밋에서 스스로* 지켜지면 체화된 것. 여기서 익힌 커밋·문서 규율은 **모든 후속 레포에서 계속 쓰인다** — M0가 #0인 이유.

⑥ test-first형 · `game-server-foundations-training` (시스템서버 입문) 1. DESIGN ⓪ → C++20(`cpp-main/`)·C#/.NET 두 트랙 17개 과제의 훈련 레포. L3 = 경계·불변식(틱 루프·lock 경계·하트비트). 2. 노트는 2편(`system-threading-channels`·`coverlet`)뿐 — 핵심은 노트가 아니라 **테스트**. 과제마다 **테스트를 먼저 읽어** 명세를 잡는다(문제지보다 테스트가 정확). 3. 덮고 구현 → 루트 `make test`(CMake/CTest + `dotnet test` + 문서검사를 한 명령) green → 답지 diff. 4. 완료 바: 테스트 green + L3(불변식)를 백지에서 설명되면 다음.

Part 4 · 분야별 학습 가이드 ★핵심

분야(모듈)마다 **무엇이 대장이고, 노트를 어떤 순서로 읽고, 커밋 리듬·L3 집중처·검증·함정이 무엇인지**가 다르다. 들어갈 분야의 카드를 펴라. 표는 한눈 요약, 카드는 상세다.

분야	대장	리듬	장르	L3 집중처(한 줄)
M0 작업규율	linux-admin	2B	⑤	규율 체화(Git/문서 검증)
C 트랙	small-shell	2B	① (stack-sort=알고리즘)	메모리·프로세스·신호/IPC·pthread·소유권
C++ 트랙	stl-container	2B	① (ray=④)	템플릿·allocator·iterator·예외안전
시스템 서버	game-server-reliability	2B	⑥	소켓(epoll/kqueue)·동시성·경계/불변식
인프라/운영	container-stack	2B	② (nrn=③)	왜 이 정책(헬스체크·secret·Compose·DB 초기화)
JVM 백엔드 ★	backend-foundations	**2A**	①	**트랜잭션·동시성·security·멱등·outbox**
Node/TS 백엔드	pong-pong	2B	①	TS 타입 경계·WebSocket·트랜잭션·서버 권위
프론트엔드	portfolio-site	2B/2A	①	RSC 경계·데이터 패칭·상태 소유권·실패 모드
모바일	mobile-reliability	mf=2B / mr=2A	①	구조적 동시성·StateFlow·오프라인 정합·refresh 레이스·재연결
AI	(af·ar → ai-capstone)	기능 커밋	①	grounding·step-cap·평가 3층·멱등 키
캡스톤	sportsbook	§3 루프	①	돈·멱등·트랜잭션·outbox·분산 (Part 6)

M0 작업규율 — 대장 `linux-admin`

- **스택**: Git · Conventional Commits · Markdown (+ Born2beroot Linux 운영: 사용자·SSH·sudo)
- **notes 순서**: `git-cli` → `conventional-commits` → `github-workflow` → `markdown`
- **리듬/장르**: 순차(2B) · 장르 ⑤ 작업규율 체화형(재구성 아님 — 규율을 지키며 한 바퀴)
- **L3**: 가벼움 — Git/문서 검증. 여기서 익힌 커밋·문서 규율은 전 레포에서 계속 쓰인다.
- **검증**: `make` + `scripts/checkdocs`
- **함정**: 모든 경로의 #0. "답지 덮고 코드 채우기"를 기대하지 마라(문제지 2편뿐).

C 트랙 — 대장 `small-shell` (곁가지 format-printer · signal-message-bus · thread-dining · stack-sort)

- **스택**: C · 프로세스/메모리 · 시그널/IPC · pthread (+ Python)
- **notes 순서**: small-shell `c-language`→`gnu-readline` · format-printer `ar`→`c-test-harness` · signal `posix-signal` · thread-dining `posix-pthread`→`posix-time-io`→`awk` · stack-sort `python`
- **리듬/장르**: 순차(2B) · **docs/notes 없음**(막히면 루트 notes 1부로 대체) · 대다수 장르 ①
- **L3**: **소유권**("누가 할당하고 누가 닫나"를 소리 내 말하기), fd·프로세스·시그널 안전, pthread
- **검증**: `make test` + 수동 시나리오
- **함정**: stack-sort는 알고리즘 — 독립 적용기(Python)로 출력 검증하는 법 자체가 배울 거리

C++ 트랙 — 대장 `stl-container` (곁가지 ray-scene-tracer)

- **스택**: C++98 · 템플릿 · allocator · iterator · 예외 안전 (+ 그래픽스)
- **notes 순서**: stl-container `cpp98` · ray-scene-tracer (노트 없음 — 핸즈온)
- **리듬/장르**: 순차(2B) · stl=장르 ① / ray=장르 ④ 따라만들기(reference-impl 없음)
- **L3**: 예외 안전 · iterator 무효화 · 템플릿 · allocator(construct 예외·rebind)
- **검증**: stl=std와 행동 대조, ray=체크섬 결정성
- **함정**: ray는 새 스택이 아니라 적용 연습 — C++ 습득은 stl-container가 담당

시스템 서버 — 대장 `game-server-reliability` (C++20·C#/.NET) (곁가지 irc-relay-server C++17 · game-server-foundations)

- **스택**: C++17/20 · Boost.Asio · 소켓(epoll/kqueue) · C#/.NET · ASP.NET
- **notes 순서**: gsr(두꺼움) `cpp20`→`csharp-dotnet8` · `cpp-compiler`→`cmake` · `boost-asio`→`spdlog`→`aspnet-core` ·

`googletest` → `ctest` → `sanitizers` · `xunit` → `fluentassertions` · irc-relay(C++17) `cpp17` → `posix-bsd-sockets` → `gnu-make` → `e2e-smoke-harness` · gsf `system-threading-channels` → `coverlet`

- **리듬/장르**: 순차(2B) · 장르 ⑥ test-first
- **L3**: 이벤트 루프(kqueue/epoll) · backpressure · heartbeat/idle · 단일 락 ownership · 경계/불변식(틱·lock·하트비트)
- **검증**: **테스트가 채점기.** 방법이 다르다 — **테스트를 먼저 읽고** 구현(문제지보다 테스트가 정확한 명세)
- **함정**: C++98→17→20 점프는 경사 동선(대안: 대장-first로 reliability·irc 먼저)

인프라/운영 — 대장 `container-stack` (곁가지 network-routing-notes)

- **스택**: Docker · Compose · Nginx · POSIX shell (+ 서브넷/라우팅)
- **notes 순서**: container-stack 셸 `posix-shell` → `apt` → `curl` → `gosu` · 컨테이너 `docker` → `dockerfile` → `yaml` → `docker-compose` · 웹 `nginx-config` → `openssl` → `cgi-fcgi` → `php-fpm-pool` → `wordpress` → `wp-cli` · DB `mariadb-install-db` → `mariadb-config` → `mysqladmin` · network-routing-notes (노트 없음 — 개념/짧음)
- **리듬/장르**: 순차(2B) · cs=장르 ②(문제지 18 → 코드처럼 재구성) / nrn=장르 ③ 손계산
- **L3**: 구현이 아니라 **왜 이 정책인가** — 헬스체크 체인 · secret 분리 · 진입 경계 · Compose 네트워킹 · DB 초기화
- **검증**: cs=정적 검증기 + 실제 기동, nrn=손계산 ↔ 검증 스크립트
- **함정**: 통합 검증은 Docker 필요(없으면 정적까지). nrn은 코드 아님 — 백지에서 라우팅 재계산

JVM 백엔드 ★주력 — 대장 `backend-foundations` (곁가지 backend-reliability)

- **스택**: Java 21 · Spring Boot · Web MVC · Validation · Security · JPA · Redis · 동시성 (+ Go · Gradle Kotlin DSL)
- **notes 순서**: bf(두꺼움) 언어 `java-21` → `jdk-21` · 빌드 `gradle` → `spring-boot-gradle-plugin` · 코어 `spring-boot` → `jakarta-servlet-api` → `spring-web-mvc` · 기능 `validation` → `security` → `spring-data-commons-pagination` · 테스트 `junit5` → `mockito` → `spring-security-test` → `testcontainers` · `go-toolchain` · br(곁가지) `gradle-kotlin-dsl` · `jakarta-persistence` → `spring-data-jpa` → `h2-database` → `spring-transaction` · `redis` → `spring-data-redis` · `go-net-http`
- **리듬/장르**: **triplet(2A)** · 장르 ① · docs/notes 보유
- **L3**: **돈·멱등·트랜잭션·outbox·동시성·security** — 이 코퍼스의 심장. **2회독.** 답지의 "왜" 절을 빼먹지 말 것
- **검증**: 테스트 채점기 / `mvn test` (또는 `./mvnw` ·Gradle)
- **함정**: PostgreSQL은 JPA로 충당(경로 A). bf는 트랜잭션·security, br은 동시성·롤백에 집중

Node/TS 백엔드 — 대장 `pong-pong` (공개·입문 대장 · 풀스택; 곁가지 grounded-travel · chatbot-evaluation)

- **스택**: TypeScript · Node · Fastify · WebSocket · PostgreSQL(풀스택). pong이 **새로 가르치는 핵심 = PostgreSQL·WebSocket**
- **notes 순서**: 입문 `pong/docs/notes/systems/node-typescript.md` (자족) → 루트 `nodejs` → `tsx` · `fastify-cors` → `fastify-cookie` → `fastify-websocket` → `ws` · `postgresql` → `pg` → `kysely` · `caddy` · `playwright` . **깊은 곁가지**: TS 언어 = grounded `typescript` → `zod` (프론트의 선행이기도), Fastify/SQLite = chatbot
- **리듬/장르**: 순차(2B) · 장르 ①
- **L3**: TS 타입 경계 · WebSocket · 트랜잭션 · **서버 권위 모델**(pong: 입력 신뢰·tick 루프·인증 전 buffer)
- **검증**: `pnpm dev` / `npm run dev` **실행하며** + 테스트(Vitest·smoke·Playwright)
- **함정**: 공유 계약(schemas)이 중심 — 화면↔코드 왕복. pong 85커밋 전부 재구성은 과투자 — L3만 깊게. **선행은 M0뿐**(front/back foundations 불필요)

프론트엔드 — 대장 `portfolio-site` (공개·자족 입문 대장 · Next.js; 곁가지 frontend-reliability · frontend-foundations)

- **스택**: Next.js · React · Tailwind v4 (+ React 심화: RSC · Router · Query · Test)
- **notes 순서**: 입문 `portfolio/docs/notes/systems/react.md` (자족) → `nextjs` · `tailwindcss-v4` . **깊은 곁가지**: React 심화 = frontend-reliability `react` → `react-server-components` → `react-router-dom` · `tanstack-react-query` → `msw` · 테스트 `vitest` → `testing-library` → `jest-axe` · `storybook` ; Zustand 워밍업 = frontend-foundations `zustand`
- **리듬/장르**: portfolio=순차(2B) / frontend-reliability=triplet(2A) 경향 · 장르 ①
- **L3**: **심화 신뢰성은 frontend-reliability가 본대** — 실패 모드(스테일 응답·낙관적 업데이트 롤백·에러 경계·RSC 경계)·데이터 패칭·상태 소유권. (입문 자족은 portfolio)
- **검증**: 테스트 채점기 + `check:repo`
- **함정**: frontend-foundations의 카탈로그-범프류 반복 커밋은 묶어서 빠르게 — 전부 같은 무게로 돌지 말 것

모바일 — 대장 `mobile-reliability` (곁가지 mobile-foundations · 캡스톤 sportsbook/mobile-client)

- **플랫폼 = Android(Kotlin + Jetpack Compose).** Kotlin은 JVM이라 Java 21·Spring·Gradle Kotlin DSL과 **최대 전이**, 캡스톤은 같은 도메인 모델 재기술.

"해외 베팅: 백엔드+게이트웨이+Android" 단일 서사가 같은 고용주 카테고리. (Flutter=발산·콜드스타트, iOS=시너지 0 → 탈락)

- **스택**: Kotlin · Compose · Coroutines/Flow · Hilt · Retrofit/OkHttp · Room · WebSocket/STOMP · JWT refresh
- **notes 순서**: mf(입문) `kotlin` → `android-sdk` → `jetpack-compose` → `compose-navigation` → `viewmodel-stateflow` → `kotlin-coroutines` → `retrofit-okhttp` → `room` → `hilt` → `material3` → `junit-mockk` → `compose-ui-test` → `gradle-android-kts` · mr(대장) `kotlin-flow` → `structured-concurrency` → `room-offline-first` → `workmanager` → `paging3` → `datastore-security` → `okhttp-websocket-stomp` → `retry-backoff` → `jwt-auth-interceptor` → `problem-detail-mapping` → `compose-performance` → `turbine` → `mockwebserver` → `espresso` → `screenshot-testing` → `hilt-testing`
- **리듬/장르**: mf=순차(2B) / mr=triplet(2A) 경향 · 장르 ①
- **L3**: 구조적 동시성/coroutine 취소 · StateFlow 상태 소유권 · 오프라인 캐시 정합성 · 토큰 refresh/인증 레이스 · WebSocket 재연결 + 멱등
- **검증**: JVM 단위(JUnit/MockK/**Robolectric**)는 SDK 없이 실행. Compose/Espresso 계측은 에뮬레이터 필요(ENV-LIMIT)
- **함정**: SDK 없는 머신에선 빌드 불가 — 코드 리딩+JVM 테스트로 돌고, 빌드 검증은 SDK 환경에서. 선행은 M0만(Kotlin은 foundations `notes/kotlin.md` 자족 온램프)

AI — `ai-foundations` · `ai-reliability` → `ai-capstone` (5 레포)

- **스택**: RAG · 평가 게이트 · 신뢰성 게이트웨이 (LLM 애플리케이션)
- **리듬/장르**: 과제/기능 커밋 · 장르 ①
- **L3**: grounding 게이트 · step-cap · 평가 3층(결정·judge·회귀) · 멱등 키 = 의도의 함수
- **검증**: 전부 **오프라인·결정적**(포트 주입) — 어디서든 실행됨
- **함정**: ai-capstone 순서 = `shared-contracts → rag-agent → reliability-gateway → eval-gate → orchestration`

캡스톤 — `sportsbook` (Part 6 상세)

- 코어 = 종합 적용(기존 139개 노트 재사용) + 신규 = 분산·운영 스택(Tier-1 8노트). 9개 레포 순서 고정.
- 모바일 클라이언트는 모바일 페어 + sportsbook 백엔드 완료 후. → **Part 6.**

Part 5 · 전체 순서 — 사슬 · 경로 프리셋

5.1 의존 사슬 — 강한 사슬은 3개뿐

```
[사슬 1]  C 트랙 → C++ 트랙 → 시스템 서버
[사슬 2]  Node/TS(grounded) → 프론트엔드
[사슬 3]  모바일 foundations → 모바일 reliability

캡스톤(sportsbook) ⟵ JVM 백엔드 + 인프라/운영   (타이트하게 좁힌 선행)
sportsbook 모바일 ⟵ 모바일 페어 + sportsbook 백엔드
```

느슨해진 의존(불필요): ~~container-stack ← small-shell~~ (POSIX shell은 가볍게만), ~~sportsbook ← pong-pong~~ (PostgreSQL은 Spring Data JPA로 충당).

5.2 경로 프리셋 A~F — 목표에 맞춰 고른다

경로	정체	순서
A. JVM 취업 최단 ★기본	목표 직결, ~5개	linux-admin → backend-foundations → backend-reliability → container-stack → **sportsbook** (network-routing 선택). C/C++/시스템/Node/프론트 **전부 생략**, PostgreSQL=JPA
B. 정통 CS 완주	모든 모듈 순차(구 단일순서)	M0 → C(5) → C++(2) → 시스템서버(3) → 인프라(2) → JVM(2) → Node/TS(3) → 프론트(3) → 캡스톤
C. 백엔드 풀폭	JVM + Node 생태계	M0 → JVM → Node/TS → 인프라 → 캡스톤
D. 풀스택 1인개발	산출물 먼저	M0 → TS(grounded) → 프론트 → JVM → 인프라 → 캡스톤
E. JVM + 모바일	해외·다양성	A 완료 후 → 모바일 foundations → 모바일 reliability → sportsbook/mobile-client
F. 42 주도 + 백/프론트 병렬	유일한 병렬	게이트(M0) → 3레인 동시(아래)

경로 A 척추(기본):

```
linux-admin ⟶ backend-foundations ⟶ backend-reliability ⟶ container-stack ⟶ sportsbook
  (M0·⑤)            (①·2A)                (①·2A)               (②·2B)       (캡스톤·순서고정)
```

경로 F 병렬 3레인(42 주도):

```
게이트: linux-admin(M0 = 42 첫 과제 = 공통 게이트)
   ├─ 레인 SPINE(42/CS): small-shell → [C 곁가지] → stl-container → ray → irc → game-server-*
   ├─ 레인 BACK(JVM):    backend-foundations → backend-reliability → container-stack → sportsbook
   └─ 레인 FRONT(TS):    grounded-travel(TS) → frontend-foundations → frontend-reliability → portfolio-site
```

원칙: FRONT는 grounded TS가 선행(사슬 2). 캘린더·peak 분산표는 parallel-track.md.

Part 6 · 캡스톤 — sportsbook

진입 체크: JVM(backend-foundations·reliability) + 인프라(container-stack) 완료 시 자연스럽다.

sportsbook은 **9개 레포**(서비스 7 + gateway + orchestration)로, **두 갈래**로 돈다:

- **(A) 코어 = 종합 적용.** 각 서비스(betting·wallet·settlement·risk·odds-feed·admin-api·shared-protocol)는 자기 `docs/commits/` 로 **Part 2 루프 그대로** 재구성. in-process Spring 코어는 **기존 139개 노트 재사용**(신규 학습 아님).
- **(B) 신규 = 분산·운영 스택.** sportsbook을 분산 시스템으로 만드는 횡단 계층은 139개에 없다. `sportsbook/orchestration/notes/` 의 **Tier-1 노트 8개**(4부 포맷 + reference-impl)를 아래 순서로 정독(=Step 1):

```
maven → spring-kafka → avro → flyway → resilience4j(+bucket4j)
      → spring-cloud-gateway → oauth2-resource-server → observability
```

순서 고정(서비스): `shared-protocol → betting → wallet → risk·odds-feed → settlement → gateway·admin → orchestration`. 서비스 들어가기 전 해당 Tier-1 노트 1편 선행.

보조 자원: `orchestration/docs/architecture/decisions`(ADR) · `chaos/` · `e2e/` · `observability/`. 통합 검증은 Docker 필요(없으면 단위까지, e2e는 답지로).

출구: 9개 레포 통합 동작 확인 → GitHub push(커리어 산출물).

6.1 모바일 캡스톤 — `sportsbook/mobile-client`

진입: 모바일 페어(foundations·reliability) + sportsbook 백엔드 완료. 코어 = 종합 적용(페어 notes 재사용), 신규 = **살아있는 분산 백엔드 클라이언트 통합** 2~3개(`stomp-over-spring-broker` · `oauth2-refresh-against-gateway` · 선택 `idempotent-bet-contract`).

소비할 API 계약 (gateway `/api/v1/*`, `/ws/v1/*` — 빌드 세션 실소스 검증):

기능	엔드포인트	인증	신뢰성 포인트
이벤트	`GET /api/v1/events[/{eventId}]`	공개	offline-first, 페이징
odds	`GET /api/v1/odds/{eventId}/{marketId}/{selectionId}`	공개	캐시 + 실시간 갱신 병합
베팅 접수	`POST /api/v1/bets`	JWT	**Idempotency-Key**, RFC 7807, 낙관적+롤백, "odds 변경→재확인"
내 베팅	`GET /api/v1/bets[/{betId}]`	JWT	페이징, 라이브 갱신
지갑	`GET /api/v1/wallet/balance`	JWT	token subject 제약
실시간 odds	`STOMP /ws/v1/odds → /topic/odds/{eventId}`	—	**재연결/백오프**, lag 처리
베팅 상태	`STOMP /ws/v1/bets → /user/queue/bets`	JWT(CONNECT)	라이브 정산, 상태 반영

인증	RS256 JWT + refresh (발급자 없음·gateway 검증만)	—	**OkHttp Authenticator 단일비행 refresh**, Keystore 보관

정합 교훈: `/ws/v1/*` 는 핸드셰이크 엔드포인트(구독 목적지는 `/topic/...`·`/user/queue/...`), 경로 파라미터 `{userId}` 없음(JWT `sub` 로 해소), 시스템에 발급자 없음(gateway=검증 전용), "market 목록" 엔드포인트 없음(selection은 odds 스트림으로 발견). 도메인 모델(Money·Odds·BetSlip…)은 `shared-protocol` 을 **Kotlin DTO로 재기술**(의존 아니라 재기술 — 클라는 JSON만).

Part 7 · 완주 & 면접

7.1 완료 기준

한 단위(커밋/exercise) "완료" — 셋을 모두 만족: ① 해당 exercise의 Completion Criteria 충족 ② 검증 스크립트 통과(`scripts/checkdocs/`·`Makefile`·CI) ③ 그 단위의 **L3 항목 백지 설명 통과**(Step 4).

프로젝트 "완료": 모든 비-merge 커밋을 Part 2 루프로 돌았고, 그 프로젝트의 **L3 집합을 백지로 설명** 가능.

경로 A 체크리스트:

```
[ ] M0  linux-admin            (Step1✓ Step2✓ L3✓)
[ ] JVM backend-foundations    (트랜잭션·security L3 백지?)
[ ] JVM backend-reliability    (동시성·롤백 L3 백지?)
[ ] INF container-stack        (Compose 기동·DB 초기화 재현?)
[ ] CAP sportsbook 코어(9서비스) + Tier-1 8노트 + 9레포 통합·push
```

전 과정 "완주" = 이 둘이 서면 끝: - **캡스톤 sportsbook 통과** — 9레포 순서 고정 + Tier-1 8노트 + 통합·push + L3(돈·멱등·트랜잭션·outbox) 백지. - **아래 12 주제축을 백지로 설명** — 전 레포 L3가 한 입으로 나온다.

7.2 면접 — 12 주제축(폭발 반경)

전 프로젝트의 **L3만 모으면** 스택을 가로질러 12개 축으로 반복된다. 이걸 백지 설명 세트로 연습한다 (각 항목의 `[레포 NNN]` 포인터 + 전 레포 370항 색인은 부록 INTERVIEW.md 정본):

축	주제	축	주제
A	동시성·락	G	시스템·프로세스·메모리
B	트랜잭션·정합성	H	인증·보안
C	멱등성·중복 제거	I	신뢰성 패턴(circuit breaker·token bucket·fail-open/closed)
D	돈 계산(minor units·BigDecimal 함정)	J	프론트·클라이언트 신뢰성(스테일·낙관적·오프라인)
E	분산·이벤트(outbox·partition key·Saga)	K	관측성·검증 문화
F	네트워크·실시간(이벤트 루프·backpressure·STOMP)	L	AI 신뢰성(grounding·step-cap·eval 3층)

7.3 지금 시작 — 첫 한 걸음

1. **첫 레포를 연다.** 대부분 경로의 #0은 `linux-admin`(M0·장르 ⑤), 코드 재구성형 본대는 `backend-foundations`(JVM 대장). **들어가면 DESIGN ⓪로 장르부터 식별**하고 Part 4 카드 + Part 3 장르대로 돈다.
2. **커밋마다 먼저 분류**(L3/L2/L1) → L3만 손 재구성, L1은 읽고 통과. 다이얼을 공격적으로 돌려라.
3. **막히면 가끔 모드 B로 변주** — AI 구현 → 답지 diff → 결함 찾기.
4. 레포 닫을 때마다 INTERVIEW.md 그 줄 복기. 시간이 없으면 **경로 A**만 돌고 12 주제축 백지 연습.

> 노트를 다시 쓰고 싶은 충동은 보통 "지루한 재구성을 건너뛰고 싶다"는 신호다. 그 지루한 벽이 학습 그 자체다 — 도망치면 AI도 할 수 있는 것만 남는다. 답지에 `## 알려진 소스 결함` 절이 있으면 재구성 전에 그것부터 본다(원본의 함정 안내).

부록 · 스택 → 대장 참조표

"이 스택을 처음 배운다면 어느 레포의 노트가 대장(가장 깊은 선행)인가."

스택	대장	곁가지
C	small-shell	format-printer · signal-message-bus · thread-dining · stack-sort
C++98	stl-container	ray-scene-tracer
C++17	irc-relay-server	ray-scene-tracer
C++20 · C#/.NET	game-server-reliability	game-server-foundations
Python	stack-sort	(각 프로젝트 테스트)
Java/Spring	backend-foundations	backend-reliability
Go · Gradle Kotlin DSL	backend-reliability	backend-foundations
TypeScript	grounded-travel	chatbot · pong · portfolio · frontend-*
JavaScript ESM	frontend-reliability	(각종 .mjs)
React	frontend-reliability	frontend-foundations · portfolio · grounded · pong
Next.js	portfolio-site	frontend-* · pong
Fastify	chatbot-evaluation	grounded · pong
PostgreSQL	pong-pong	backend-reliability
SQLite	chatbot-evaluation	grounded
Docker/Compose · POSIX shell	container-stack	chatbot · grounded · pong · small-shell
Kotlin · Jetpack Compose · Android · Coroutines/Flow · Room	mobile-reliability	mobile-foundations · sportsbook/mobile-client
WebSocket/STOMP(client)	mobile-reliability	sportsbook/mobile-client
Git/Markdown	linux-admin	전 프로젝트